

EVERYTHING PROMPT ENGINEERING

The Junior Dev AI Survival Kit

What every fresh developer, undergrad, and interview candidate needs to know about working with AI in 2026. The terms, the habits, the red flags, and the things nobody is telling you.



2026 Edition

A free guide from **Everything Prompt Engineering**

CONTENTS

What's Inside

- 01 The 5-Minute AI Vocabulary Test**
12 essential terms in plain English

- 02 The 9-Point Self Code Review**
Catch your own bugs before standup

- 03 Red Flags in AI-Generated Code**
What signals 'I didn't actually write this'

- 04 The Portfolio Project Test**
Three questions that separate callbacks from rejections

- 05 How to Talk About AI-Assisted Code in Interviews**
Honesty without undermining yourself

- 06 Six Honest Things Worth Knowing**
What this kit refuses to lie about

SECTION 01

The 5-Minute AI Vocabulary Test

If you can't explain these terms in your own words, here are the explanations.

Most AI vocabulary sounds harder than it is. The terms below come up in interviews, in standup meetings, and on job descriptions. Each one is defined in plain English, with one sentence on why it matters.

Prompt

What gets typed into the AI. The actual text of the request.

The same AI gives wildly different answers depending on how the prompt is written.

System Prompt

The instruction that shapes the AI's behavior for an entire conversation, set once at the start.

It's where most production AI features earn their personality and reliability.

Context Window

How much text the AI can read in a single request, total, including everything.

Long conversations or large documents will eventually hit this limit.

Token

Roughly 4 characters of English text. The unit AI providers charge by.

Cost and length limits are both measured in tokens, not words.

Embedding

A numerical representation of text that lets machines compare meaning.

Embeddings power semantic search, RAG, and most AI features that need to find 'related' content.

RAG (Retrieval-Augmented Generation)

An architecture that retrieves relevant text from a database and passes it to the AI at query time.

It's how most AI features access information the model wasn't trained on, like company docs or user data.

Vector Database

A database optimized for storing and searching embeddings.

RAG systems need one. Common options include Pinecone, pgvector, and Weaviate.

Fine-Tuning

Training a model further on a specific dataset to specialize its behavior.

Most teams don't actually need it. Prompting and RAG handle most use cases more cheaply.

Agent

A language model that can request to use tools and take actions, not just generate text.

It's the architecture behind many production AI features, from coding assistants to customer support.

Function Calling / Tool Use

The mechanism that lets a model request actions from your code.

It's how agents actually work under the hood. The model decides; your code executes.

Hallucination

When an AI generates something that sounds confident but is factually wrong.

It's the most common failure mode of AI features in production. Designing around it is half the job.

MCP (Model Context Protocol)

A standard for connecting AI models to external tools and data sources.

It's becoming the common interface for letting AI tools talk to other systems.

THE HONEST TAKE

These twelve terms cover most production AI conversations in 2026. Anyone who can explain all twelve in their own words has more AI vocabulary than the average junior developer. The bar is genuinely lower than it looks.

SECTION 02

The 9-Point Self Code Review

Catch the embarrassing bugs in your own pull request before your senior catches them.

Open a fresh AI chat. Paste the diff or the changed files. Then run the prompts below in order. The full sequence takes about 8-10 minutes and prevents most of what seniors flag in code review.

1. Plain-English explanation

```
Walk me through this code line by line in plain English. Focus on what it does, not what it's supposed to do. Flag anything that seems off.
```

What this catches: Catches the gap between what was intended and what was actually written.

2. Hardcoded secrets

```
Are there any hardcoded API keys, database passwords, tokens, or other secrets in this code? Any environment variables that should be loaded but aren't?
```

What this catches: The highest-stakes check on the list. Hardcoded credentials are how juniors lose trust permanently.

3. Null and empty inputs

```
What happens if every input to every function is null, undefined, an empty string, an empty array, or zero? List the failure cases.
```

What this catches: AI tools love the happy path and skip edge cases by default. This forces them to think about failure.

4. Codebase consistency

```
Here's a sample file from our repo: [paste]. Does my code follow the same patterns, naming conventions, and architecture? Where does it diverge?
```

What this catches: Prevents the 'this code looks like it came from a different team' feedback in review.

5. N+1 queries

Are there any database queries inside loops? Any places where I'm fetching one record at a time when I could fetch them all at once?

What this catches: AI's most common backend mistake. Fine in dev, brutal in production.

6. Missing test cases

What are the 5 test cases for this code that I'm probably missing? Focus on edge cases, error states, and unusual inputs.

What this catches: The fifth test case is always the one your senior asks about in review.

7. Debug code left behind

Are there any console.log, print, debugger statements, commented-out code, TODO comments without context, or temporary variables I forgot to clean up?

What this catches: Most embarrassing because most easily fixed. Always run this prompt.

8. Naming clarity

Are any of my variable, function, or class names confusing, misleading, or inconsistent? Suggest better names where they apply.

What this catches: The single thing seniors comment on most often. Naming is what affects everyone reading the code later.

9. Production failure handling

If this code throws an unexpected error in production, what would the user see? Is it logged? Does it crash anything else? Are errors handled gracefully?

What this catches: The senior-engineer question. Separates 'works' from 'safe to ship.'

SECTION 03

Red Flags in AI-Generated Code

What signals to a reviewer that you didn't actually understand what you submitted.

Senior engineers can usually tell within thirty seconds whether code was understood by the person who submitted it. The signals are specific. Below are the patterns that consistently make AI-generated code look like it came from a tutorial follower, not a developer.

1. Imports that aren't used

AI tools sometimes import libraries the code doesn't actually call. Unused imports signal that the code was generated mechanically rather than written deliberately.

2. Comments that don't match the code

When AI generates a function and adds a comment, the comment describes what the code is supposed to do, not what it actually does. Mismatches between comment and behavior are a giveaway.

3. Cleverness that solves the wrong problem

AI loves elegant solutions. Sometimes the elegant solution is solving a problem the codebase didn't have, or assuming a constraint that doesn't apply. Senior reviewers spot this fast.

4. Variable names that don't match the rest of the file

AI uses generic names by default. If everything else in the file uses domain-specific names (customerOrderId, invoiceTotal) and the new code uses generic ones (id, total), the inconsistency is loud.

5. Try-catch blocks that swallow errors silently

AI often wraps code in try-catch and either ignores the error entirely or returns a vague default. This is one of the worst production patterns. Always inspect what happens in the catch.

6. Functions that do too many things

AI tends to combine multiple responsibilities into a single function because the request was phrased as one task. Real codebases prefer single-responsibility functions.

7. Code that doesn't match the language's idioms

Python code that looks like Java. JavaScript that looks like Python. AI sometimes ignores idioms that are obvious to anyone fluent in the language. Reviewers notice immediately.

8. References to APIs or libraries that don't exist

The most dangerous pattern. AI occasionally invents method names, import paths, or library features that simply don't exist. Always verify imports and method calls against current documentation.

HOW TO USE THIS LIST

Before submitting any AI-assisted code, scan for these eight patterns. Most code reviews that go badly trace back to one of them. The fix is always the same: read every line, understand what it does, and rewrite anything that doesn't match the rest of the codebase.

SECTION 04

The Portfolio Project Test

Three questions that separate the projects that get callbacks from the ones that don't.

Recruiters in 2026 spend less than ten seconds per portfolio on first review. These three questions reveal which projects earn a second look, and which get skipped.

Question 1

Does this project solve a real problem that someone (preferably not just me) actually had?

The strongest portfolio projects come from real friction. A budget tracker built because freelance income was unpredictable. A scraper built because a website lacked a feature. A tool a family member actually uses. The answer to this question is the project's story, and stories are what recruiters remember.

Question 2

Could I describe what this project does to a non-technical person in two sentences?

If the answer is no, the project is probably about technology, not about a problem. 'A React + Tailwind + Firebase app with OpenAI integration' is a tech stack. 'A tool that helps freelancers track unpredictable monthly income' is a project. Recruiters fund the second kind.

Question 3

Is the README the first thing a hiring manager would see, and does it start with the problem?

Strong READMEs open with a sentence like 'I built this because...' Weak READMEs open with installation instructions or a feature list. The README is often the only piece of the project a recruiter actually reads. Treat it as the front door, not the documentation.

QUICK DIAGNOSTIC

If a portfolio project can't pass all three questions, it's probably hurting the portfolio more than helping it. Three excellent projects that pass all three beat twelve mediocre projects every time. When in doubt, delete.

SECTION 05

How to Talk About AI-Assisted Code in Interviews

Honesty about AI use, without lying or undermining yourself.

Most candidates in 2026 used AI tools to build their portfolio projects. Most don't know how to talk about that without either hiding it (which fails when interviewers probe) or oversharing (which makes the work sound less impressive than it was).

The framing below works because it's honest, specific, and matches how senior engineers actually use AI on the job. Hiring managers respond well to it because it sounds like a future colleague, not a candidate trying to game the question.

When asked: 'Did you write this code yourself?'

Weak answer: 'Yes, all of it.' (Then the interviewer probes a function and the candidate can't explain it.)

Better answer: 'I designed the project and made the architectural decisions. I used AI tools for parts of the implementation, the same way I'd use them on the job. I can walk you through any part of it.'

Why this works: It claims the right things (judgment, design, ownership) without claiming things that won't survive scrutiny (writing every line by hand). It also invites the interviewer to test understanding, which is the right move because most candidates avoid it.

When asked: 'Walk me through this function.'

Weak answer: Reading the code aloud, line by line, with no commentary on why it's structured this way.

Better answer: 'This function handles X. The reason I structured it this way is Y. The trade-off I made was Z. If I rebuilt this today, I'd probably change A.'

Why this works: It demonstrates understanding (the why), judgment (the trade-off), and continued growth (what would change). All three signal a candidate who's actively building skill, not just shipping code.

When asked: 'How did AI help you build this?'

Weak answer: 'I just asked ChatGPT and it wrote it for me.' (Even when partially true, this signals no judgment.)

Better answer: 'I used AI for boilerplate, for unfamiliar library APIs, and for first drafts of test cases. I read every line before accepting it, rewrote the parts that didn't fit, and made the architectural choices myself. The result is code I understand and can defend.'

Why this works: It's specific about what AI did and didn't do. Specificity is credibility. Vague answers about AI use sound evasive. Specific answers sound professional.

THE PRINCIPLE BEHIND ALL OF THESE

Hiring managers in 2026 know candidates use AI. The question they're actually asking isn't 'did you use AI?' It's 'do you have the judgment to use it well?' Every answer above is built to demonstrate judgment, which is what's actually being evaluated.

SECTION 06

Six Honest Things Worth Knowing

What this kit refuses to lie about.

Most career advice is calibrated to be encouraging. This section is calibrated to be true. Six things worth knowing as a junior developer entering the AI era.

01

The job market is harder than it was three years ago, and that's not your fault.

Entry-level postings have dropped sharply since 2023. The dip is real, structural, and outside any individual's control. Treat the market as the weather: harder than it should be, but adjustable for. The candidates who do best are the ones who plan for it instead of taking it personally.

02

AI fluency is a real skill, and it will separate candidates this year.

It's not optional. It's not hype. Knowing how to use AI tools well, knowing their failure modes, and knowing how to talk about them in interviews are now part of the baseline expectation for technical roles. The good news: almost nobody has mastered this yet, so the bar to stand out is genuinely low.

03

Tutorial projects don't work anymore.

Weather apps, to-do lists, and Netflix clones used to be enough. They aren't, because recruiters have seen too many. The shift toward 'projects that solve real problems' is permanent. Building one good real-problem project is worth more than five tutorial projects.

04

Honesty about AI use beats trying to hide it.

Hiring managers can almost always tell when a candidate isn't being straightforward about AI use. The candidates who lose offers aren't the ones who used AI. They're the ones who pretended they didn't.

05

The skills that matter most aren't technical.

Communication, judgment, the ability to ask good questions, the discipline to verify before trusting. These have always mattered, and they matter more now that AI handles the technical baseline. Junior candidates who invest in the non-technical skills consistently outperform those who don't.

06

Most of the advice circulating right now is wrong.

Career advice from before 2024 doesn't account for the AI shift. Career advice from after 2024 often overcorrects in the other direction. The candidates who do best are the ones who think for themselves, test what they hear, and adjust based on what actually works for them.

That's the kit.

Six sections. The vocabulary, the self-review, the red flags, the portfolio test, the interview language, and the honest things.

Everything Prompt Engineering publishes for junior devs, fresh grads, and undergrads navigating AI as a real career skill.

Subscribe at arslandg.substack.com